

GOOD OBJECT DESIGN

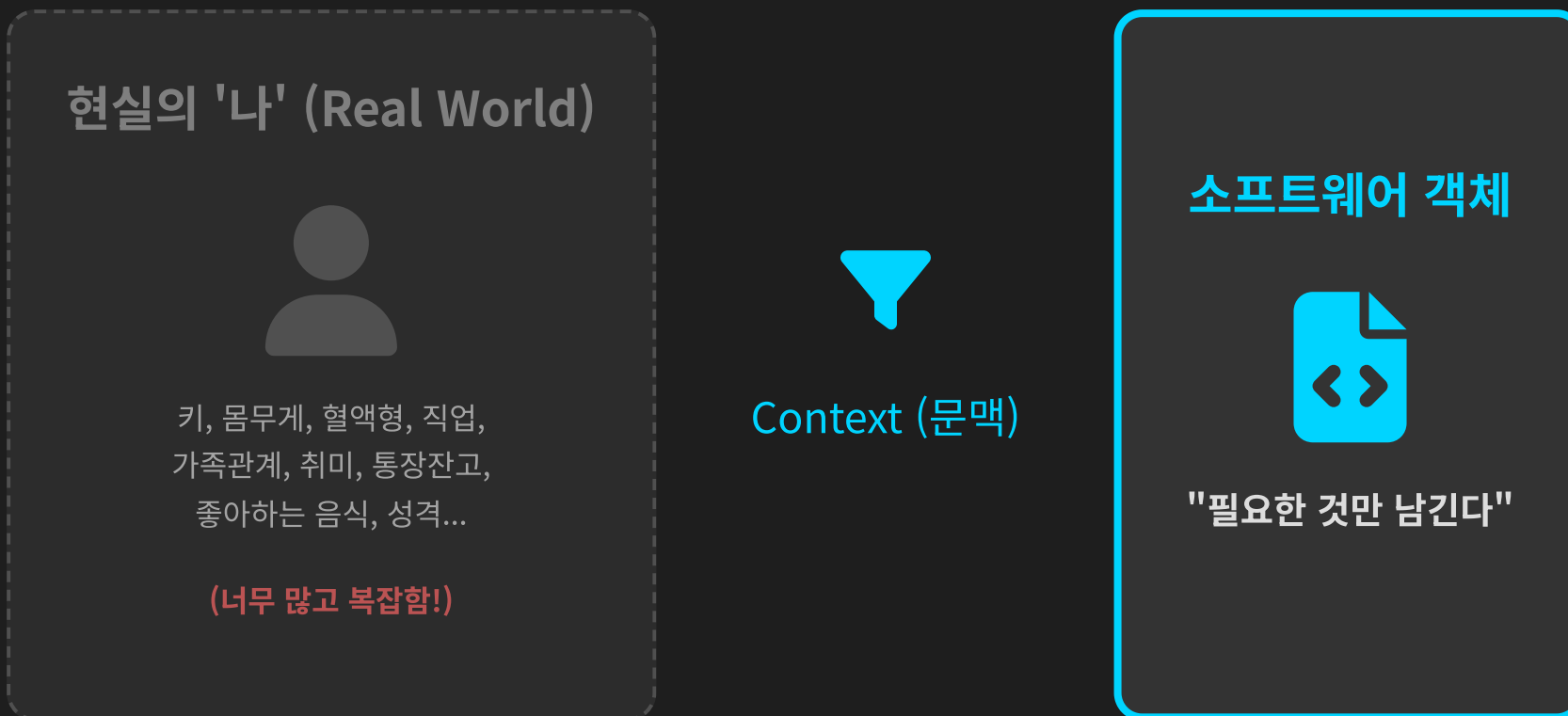
좋은 객체의 조건: 추상화와 캡슐화

"복잡함은 덜어내고, 소중한 것은 감춰라"

Chapter 6. Abstraction & Encapsulation

1. 추상화 (Abstraction): 모델링의 미학

"현실의 복잡한 객체에서 **관심 있는 특징**만 뽑아내는 과정입니다."



문맥(Context)에 따른 추상화의 차이

"똑같은 '사람'이라도, 어떤 프로그램이냐에 따라 모습이 달라집니다."



병원 시스템

Class Patient (환자)

- 혈액형 (O)
- 키/몸무게 (O)
- 통장 잔고 (X)
- 학교 성적 (X)



은행 시스템

Class Customer (고객)

- 통장 잔고 (O)
- 신용 등급 (O)
- 혈액형 (X)
- 취미 (X)

"추상화는 '무엇을 버릴지' 결정하는 과정입니다."

2. 캡슐화 (Encapsulation): 보호와 책임

"추상화로 뽑아낸 소중한 데이터, 아무나 만지게 둘 것인가?"

캡슐화 전 (Before)



데이터가 노출되어 있음

```
account.balance = -10000;
```

"누군가 잔고를 음수로 만듦!
(데이터 오염)"



캡슐화 후 (After)



데이터를 꽂꽂 숨김 (은닉)

```
account.deposit(1000);
```

"정해진 메서드로만 접근 가능!
(데이터 보호)"

왜 캡슐화를 해야 하는가?



1. 데이터 보호 (Information Hiding)

외부에서 함부로 값을 변경하지 못하게
막습니다.

(Private 변수 사용)



2. 무결성 보장 (Integrity)

"나이는 음수가 될 수 없다"

"잔고는 마이너스가 될 수 없다"

이런 규칙을 메서드 안에서 검증합니다.

```
class Account:
    # 1. 데이터 숨기기 (변수명 앞 __)
    def __init__(self):
        self.__money = 0

    # 2. 메서드를 통해서만 접근 허용 (규칙 적용)
    def deposit(self, amount):
        if amount < 0:
```

```
print("마이너스 입금 불가!")  
return  
self.money + amount
```

요약: 좋은 객체를 만드는 법

1. 추상화 (Abstraction)

"우리 프로그램의 목적(문맥)에 맞춰
불필요한 것은 버리고 핵심만 남기세요."

2. 캡슐화 (Encapsulation)

"데이터를 숨기고 메서드로 감싸세요.
나의 데이터는 내가 책임져야 합니다."

"이 두 가지가 합쳐질 때,
바로소 '믿을 수 있는 부품(객체)'이 됩니다."